

Prova finale

Ingegneria del Software 2025/2026

Funzionalità implementate

- Regole complete
- TUI e GUI: la TUI utilizza i metodi della libreria standard di Java, mentre la GUI è realizzata con Swing
- Socket e RMI
- Partite Multiple
- Classifica Partite su DB: per realizzare il database è stato utilizzato MySQL come DBMS.
- Persistenza

Divisione del lavoro

La suddivisione del lavoro è stato eseguita in modo tale da consentire una maggiore flessibilità e un'equilibrata suddivisione del lavoro tra i membri.

Qualvolta fosse stato necessario, sono stati organizzati incontri generali del gruppo per comprendere a fondo i compiti assegnati durante i laboratori, delineando per prima i caratteri generali del lavoro da svolgere, per poi eseguire una sua suddivisione in blocchi in contemporanea alla definizione dei dettagli implementativi ad alto livello relativi a ciascuna parte attraverso le idee fornite dai membri.

I resoconti degli incontri generali sono stati sintetizzati e organizzati all'interno di un workspace condiviso. In particolare, sono stati registrati in tabelle le diverse task da completare, assieme ad una descrizione contenente le idee prodotte durante la riunione e alla durata stimata del task, in modo che ciascun membro potesse decidere i propri compiti in base alla propria disponibilità e in modo che potesse svolgerli con un elevato grado di autonomia. Inoltre, per ciascun task o gruppo di task è stato cercato di realizzare branch separati su Git, per evitare conflittualità a livello di workflow e per mantenere il branch *master* il più "pulito" possibile.

Infine, a livello di coordinazione, oltre agli incontri dal vivo, sono state utilizzate varie piattaforme di comunicazione sia per gli incontri sia per coordinare lo sviluppo di parti diverse del progetto, qualora fosse richiesto.

Scelte implementative e design pattern

Visitor: è stato utilizzato per gestire l'eterogeneità degli effetti degli edifici. In particolare, sono state create due interfacce: *BuildingVisitor* e *VisitableBuilding*. La prima definisce per ciascun edificio che implementa *VisitableBuilding* un overload del metodo *visit*. *BuildingVisitor* è implementato dalla classe *BuildingHandler* che gestisce e implementa in modo centralizzato l'effetto degli edifici.

State: sono state realizzate la classe *GameState* nel model per controllare il flusso del gioco e la classe *LobbyState* nel controller per gestire le singole lobby. In particolare, in *GameState* è definito un metodo *update* per gestire i cambiamenti interni dello stato ed eventuali passaggi ad uno stato successivo e dei metodi *validate* per validare gli input da utente. Invece, *LobbyState* definisce i metodi relativi alla gestione dei player e della lobby e all'interazione tra questi (per esempio, *joinLobby* e *startLobby* hanno comportamenti diversi a seconda che la lobby sia piena o meno).

Prototype: è stato utilizzato per la generazione delle mazze di pesca: all'interno del file *deck.json* sono contenuti per ciascuna carta dei "template" insieme al numero di carte da generare. Il design pattern viene utilizzato per realizzare la logica di clonazione delle *AbstractCard* a partire dal template, per evitare riferimenti condivisi.

Builder: è stato utilizzato per la creazione e definizione delle caratteristiche degli oggetti *JPanel* di *Swing*: la classe *PanelBuilder* contiene metodi specifici per la definizione del layout, dei componenti e delle proprietà geometriche di un singolo pannello, incrementando la leggibilità del codice e la coerenza delle definizioni, parallelamente ad una riduzione di ripetizioni attraverso il riutilizzo dei metodi.

Singleton: è stato utilizzato per realizzare la classe *ServerController*, contenente i metodi relativi alla gestione delle richieste in entrata provenienti dai client, in aggiunta ai quelli di gestione delle lobby e dei client. Il singleton pattern è applicabile per l'unicità dell'istanza di *ServerController* ed è utilizzato per facilitare l'accesso ai suoi metodi dai componenti del controller a lato server, evitando un elevato numero di riferimenti salvati e/o chiamate di getters per ottenerli.

Command: è stata realizzata un'interfaccia *Command*, contenente un unico metodo *execute*. Le implementazioni di tale metodo corrispondono alle azioni eseguibili dall'utente a lato client e sono state riutilizzate per gestire le interazioni sia con UI differenti sia con schermate diverse della stessa UI.

Observer: è stato utilizzato per gestire la notifica di eventuali aggiornamenti dei dati di una lobby per tutti i client collegati ad essa (non solo i client collegati come 'giocatori', ma anche quelli in 'attesa').

Decorator: è stato utilizzato sia a lato client sia a lato server per gestire le interazioni con lo stub ottenuto con RMI. In particolare, *RMIClientInterface* e *RMIserverInterface* vengono utilizzati come contenitori dello stub e l'implementazione dei loro metodi corrisponde alla chiamata dei metodi remoti con un'appropriata gestione delle eccezioni e della sincronizzazione.

Strategy: è stato utilizzato per l'astrazione dei protocolli di rete utilizzati per la comunicazione tra client e server. In particolare, le implementazioni di *ServerInterface* e *ClientInterface* definiscono il modo concreto in cui avviene la comunicazione tra client e server (RMI o TCP) e mediante il polimorfismo ai client e al server è possibile comunicare senza necessitare di sapere qual è il protocollo effettivamente utilizzato.

Protocolli di comunicazione

Le interfacce *VirtualClient* e *VirtualServer* contengono i metodi corrispondenti alle interazioni client-server e sono implementate sia da *ServerController* e *ClientController*.

Per il protocollo TCP:

- **Reponse/Request:** classe astratta che rappresenta i messaggi inviati dal server (client) al client (server) e definisce un metodo *receive* che prende in ingresso un *ClientController* (*ServerController*). Ciascuna classe concreta che estende *Response* (*Request*) corrisponde ad un metodo di *VirtualClient* (*VirtualServer*), chiamato tramite *receive*, e contiene i dati necessari all'esecuzione di tale chiamata.
- **TCPClientInterface/TCPServerInterface:** implementa *ClientInterface* (*ServerInterface*) e viene utilizzato a lato server (client) per costruire i *Response* (*Request*) da inviare al client (server) e mette in coda a *IOTaskService* l'invio dei messaggi tramite *ClientHandler* (*ServerHandler*).
- **NetworkServer:** implementa *Thread* e ha il compito di ascoltare costantemente e accettare i tentativi di connessione da parte dei client.
- **ClientHandler/ServerHandler:** implementa *Thread* e viene creato a lato server (client) all'apertura di ogni connessione con un client (server) per gestire i messaggi in ingresso e in uscita relativi ad esso.
- **Gson:** viene utilizzato per la serializzazione e la deserializzazione dei messaggi scambiati tra server e client.

Per il protocollo RMI:

- **RMIClientInterface/RMIServerInterface:** estende *ClientInterface* (*ServerInterface*) e viene utilizzato a lato server (client) per mettere in coda a *IOTaskService* le chiamate ai metodi remoti di *ClientController* (*ServerController*) tramite *RMIClientService* (*RMIServerService*), gestendo appropriatamente i *RemoteException* lanciati dai metodi dello stub.
- **RMIClientService/RMIServerService:** estende *ClientInterface* (*ServerInterface*) e viene esportato come stub, svolgendo il ruolo di "entry-point" per le chiamate remote dal server (client). Viene utilizzato per rendere le chiamate remote RMI il meno bloccanti possibili, mettendo in coda a *CPUTaskService* i metodi di *ClientController* (*ServerController*) e sbloccando immediatamente il chiamante.

Componenti condivisi:

- **ClientInterface/ServerInterface:** classe astratta che implementa *VirtualClient* (*VirtualServer*) e ne modifica i metodi rimuovendo il lancio dei *RemoteException* (richiesti dall'interfaccia *Remote*) e dunque assicurando che le classi concrete che la estendono non lancino tali eccezioni.
- **ConnectionMonitor:** realizza la funzionalità di *Ping-Pong* tra server e client, tramite uno *ScheduledExecutorService* a lato client che invia un ping al server secondo intervalli regolari. Inoltre, sia per client sia per il server viene salvato il timestamp dell'ultimo ping proveniente dall'interlocutore in modo da disconnettersi qualora il tempo di "silenzio" superi una certa soglia.
- **IOTaskService:** virtual thread executor service utilizzato esclusivamente per eseguire parallelamente operazioni I/O-bound, in modo da gestire in modo efficace i blocchi dovuti alle operazioni di scrittura e lettura attraverso il montaggio e lo smontaggio dei thread virtuali. Sia in *ServerController* sia in *ClientController* gestiscono i thread per l'invio dei messaggi e la ricezione dei messaggi (TCP) e le chiamate remote a metodi in uscita (RMI).
- **CPUTaskService:** fixed thread pool executor service utilizzato esclusivamente per seguire parallelamente operazione CPU-bound e dimensionato con un numero di thread pari al numero di core del dispositivo meno un numero costante che dipende dalla presenza di altri executor service, in modo da evitare troppi context switch. Viene utilizzato principalmente eseguire i metodi richiesti dall'esterno tramite chiamate remote (RMI) o messaggi (TCP).

Percentuali di copertura dei test

Element ^	Class, %	Method, %	Line, %	Branch, %
it.polimi.ingsw.model	100% (47/47)	83% (352/422)	80% (948/1183)	77% (239/308)
board	100% (5/5)	65% (36/55)	54% (77/142)	52% (18/34)
card	100% (29/29)	88% (154/175)	80% (328/408)	65% (43/66)
gameState	100% (7/7)	80% (52/65)	87% (286/327)	87% (124/142)
player	100% (4/4)	83% (72/86)	78% (147/187)	78% (25/32)
BuildingHandler	100% (1/1)	100% (26/26)	100% (78/78)	83% (25/30)
Game	100% (1/1)	80% (12/15)	78% (32/41)	100% (4/4)
it.polimi.ingsw.controller.server	46% (7/15)	18% (40/211)	26% (171/657)	19% (34/172)
lobby	75% (6/8)	37% (26/70)	33% (81/240)	20% (17/84)
network	0% (0/6)	0% (0/112)	0% (0/225)	0% (0/28)
ServerController	100% (1/1)	48% (14/29)	46% (90/192)	28% (17/60)